

MODUL 3
Event, State, Imperatif-Deklaratif, dan Implisit Intent



Dosen Pengampu:

Muhammad Aminul Akbar, S.Kom., M.T.

Disusun oleh:

Dhea Akmalia Fibri - 245150407111081

PROGRAM STUDI SISTEM INFORMASI
FAKULTAS ILMU KOMPUTER
UNIVERSITAS BRAWIJAYA

2026

Percobaan

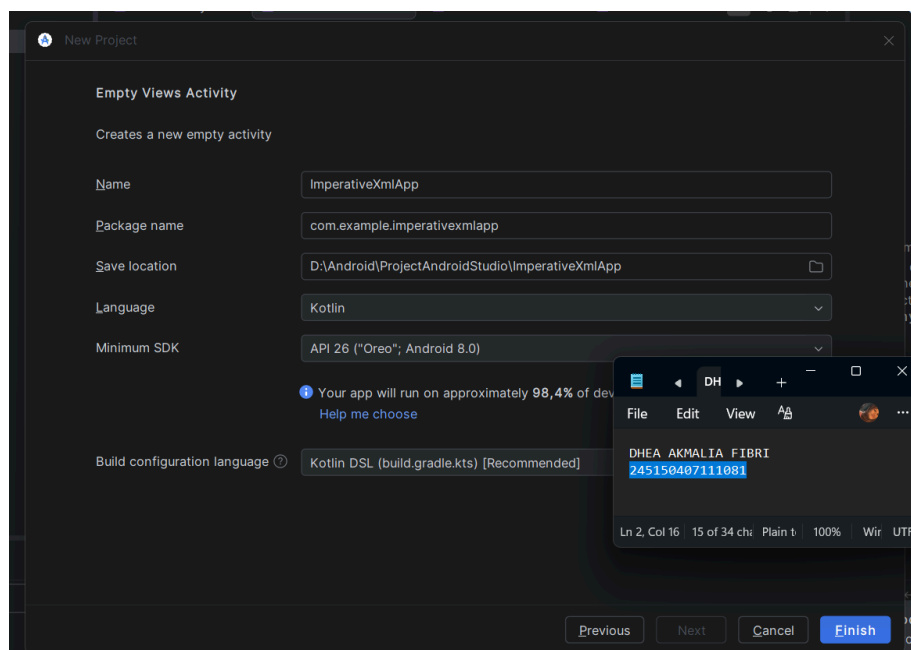
Perbandingan Imperatif dan Deklaratif dalam Perubahan Teks

Pra-Percobaan

Pada percobaan ini dibuat dua project terpisah dengan hasil akhir yang sama. Teks awal adalah "Belum diklik". Setelah tombol "Ubah Teks" ditekan, teks berubah menjadi "Tombol sudah diklik". Perbedaan yang diamati bukan hasil akhirnya, melainkan cara program memperbarui UI.

Project A - XML/View (Imperatif)

1. Buat project Android Studio baru menggunakan template Empty Views Activity. Beri nama project ImperativeXmlApp dan pilih Kotlin.



2. Buka res/layout/activity_main.xml, kemudian gunakan kode berikut:

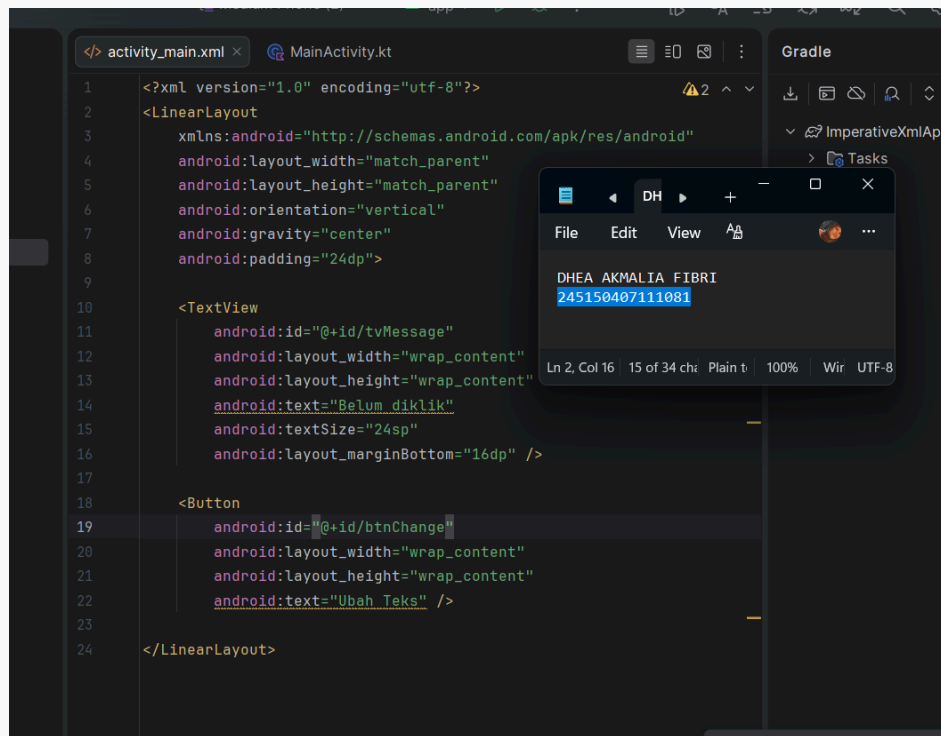
```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="24dp">
```

```

<TextView
    android:id="@+id/tvMessage"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Belum diklik"
    android:textSize="24sp"
    android:layout_marginBottom="16dp" />

<Button
    android:id="@+id/btnChange"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Ubah Teks" />
</LinearLayout>

```



3. Buka MainActivity.kt. Sesuaikan package dengan package project, kemudian gunakan kode berikut:

```

package com.example.imperativexmlapp

import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

```

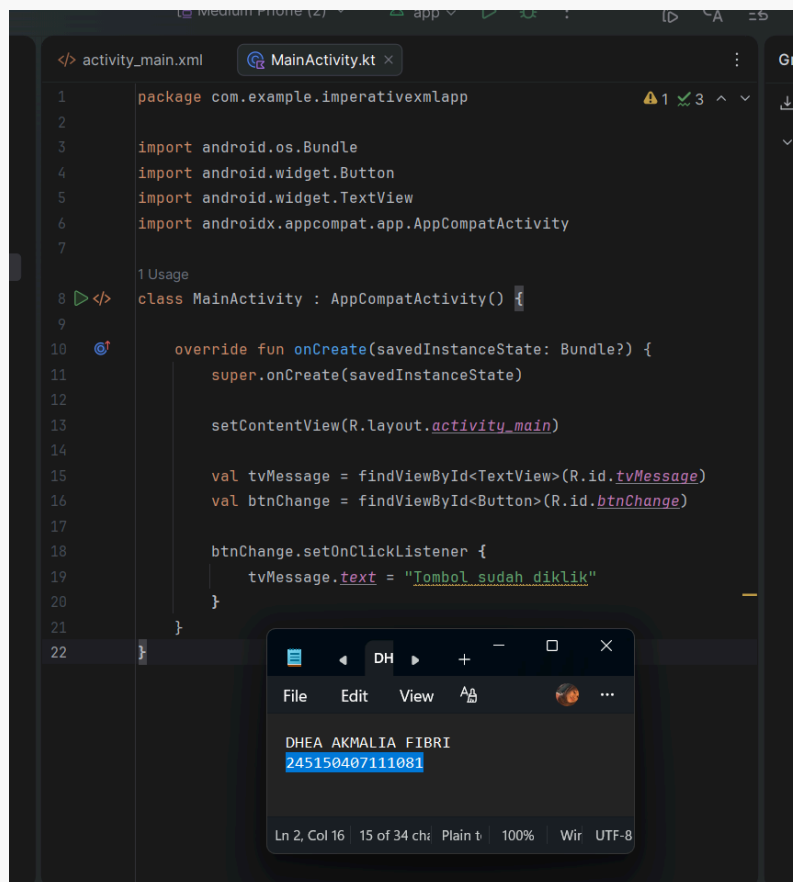
```

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)

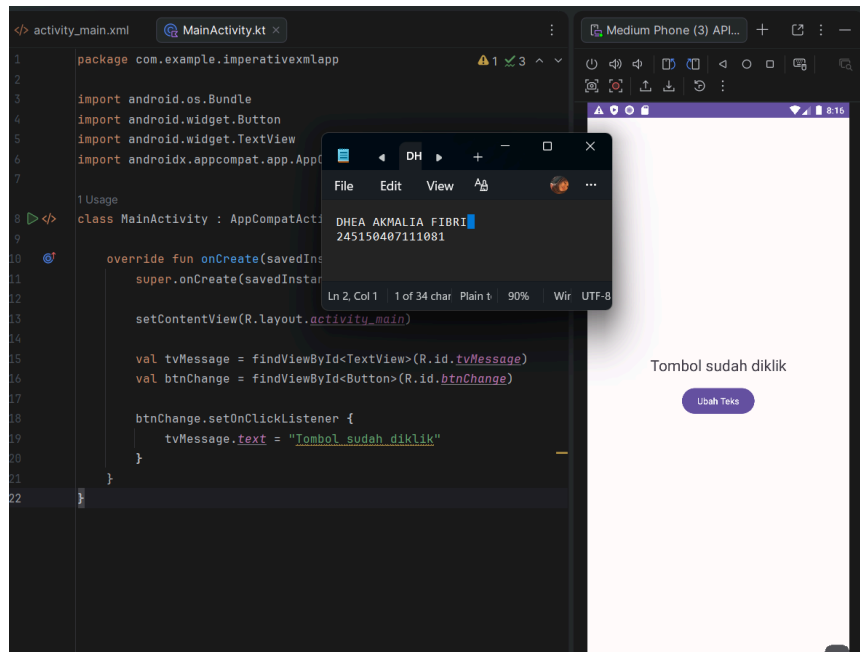
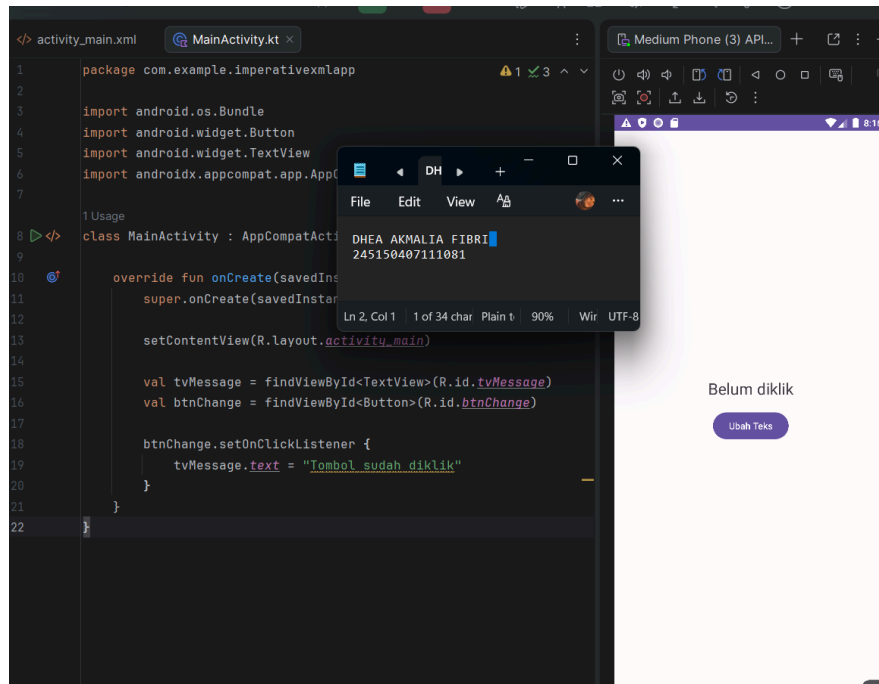
    val tvMessage = findViewById<TextView>(R.id.tvMessage)
    val btnChange = findViewById<Button>(R.id.btnChange)

    btnChange.setOnClickListener {
        tvMessage.text = "Tombol sudah diklik"
    }
}

```

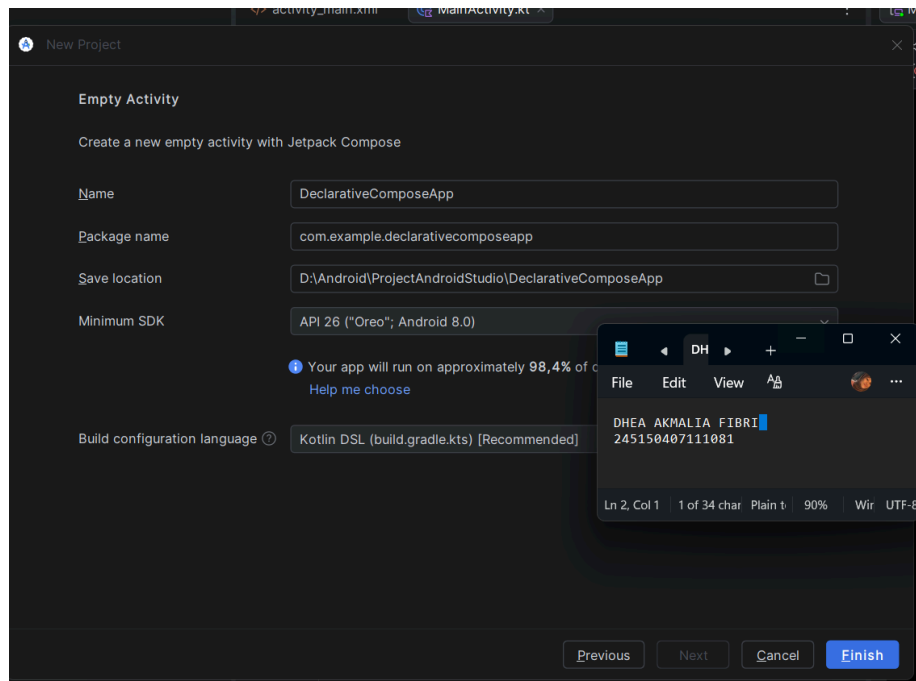


4. Jalankan aplikasi. Pastikan teks berubah setelah tombol diklik.



Project B - Jetpack Compose (Deklaratif)

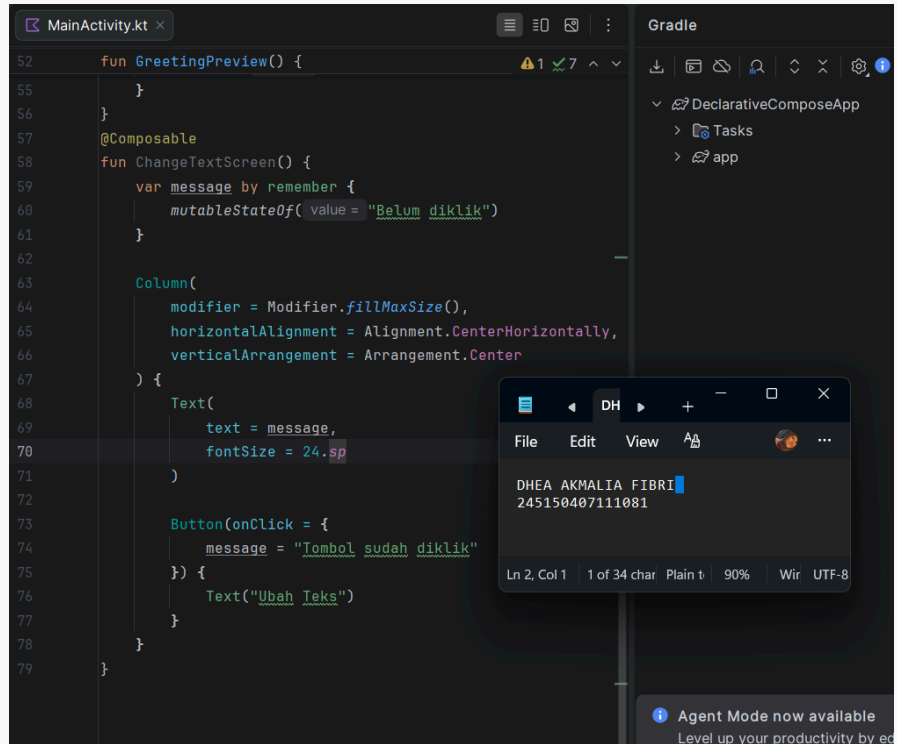
1. Buat project Android Studio baru menggunakan template Empty Activity untuk Jetpack Compose. Beri nama project DeclarativeComposeApp dan pilih Kotlin.



2. Buka MainActivity.kt, pertahankan package project, kemudian gunakan fungsi composable berikut:

```
@Composable
fun ChangeTextScreen() {
    var message by remember {
        mutableStateOf("Belum diklik")
    }

    Column(
        modifier = Modifier.fillMaxSize(),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Text(text = message, fontSize = 24.sp)
        Button(onClick = {
            message = "Tombol sudah diklik"
        }) {
            Text("Ubah Teks")
        }
    }
}
```

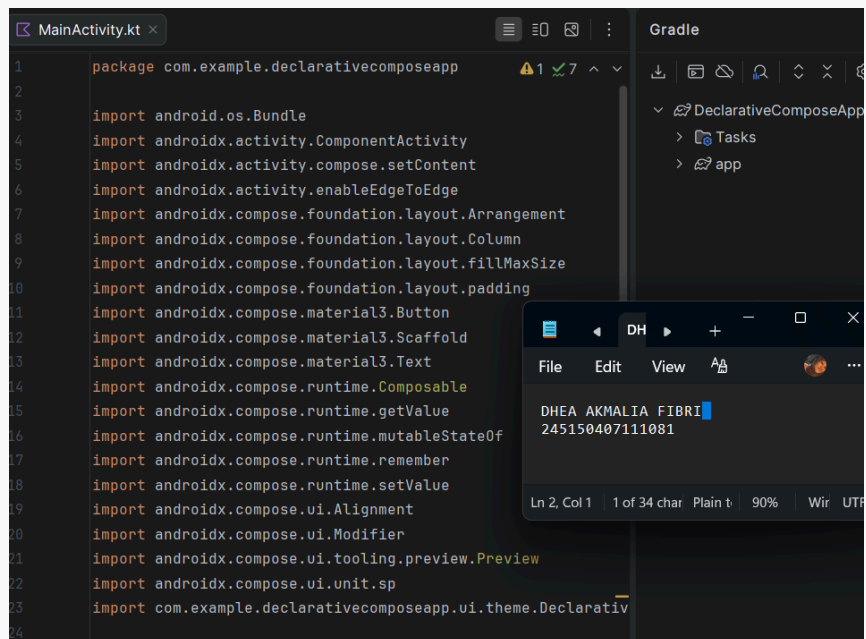


3. Tambahkan import yang diperlukan:

```

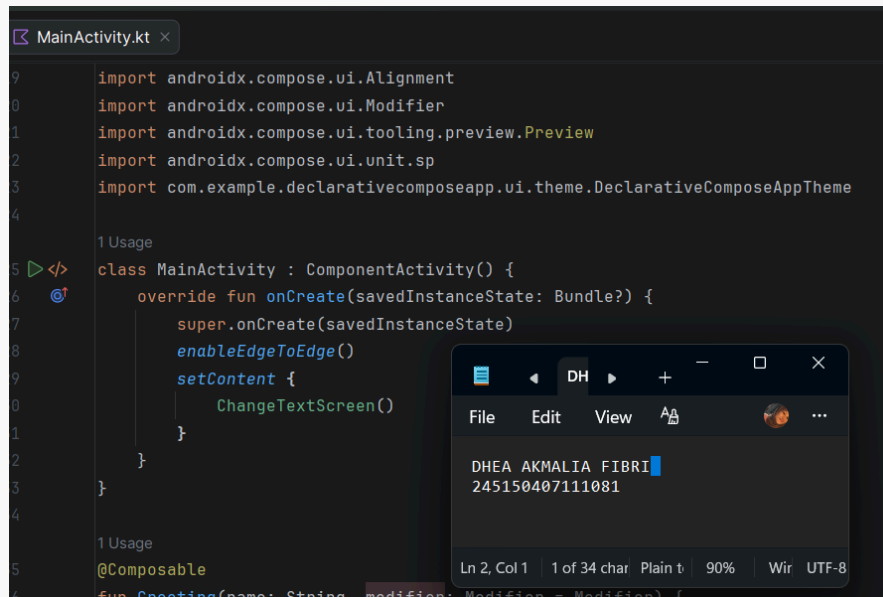
import androidx.compose.foundation.layout.*
import androidx.compose.material3.Button
import androidx.compose.material3.Text
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.sp

```

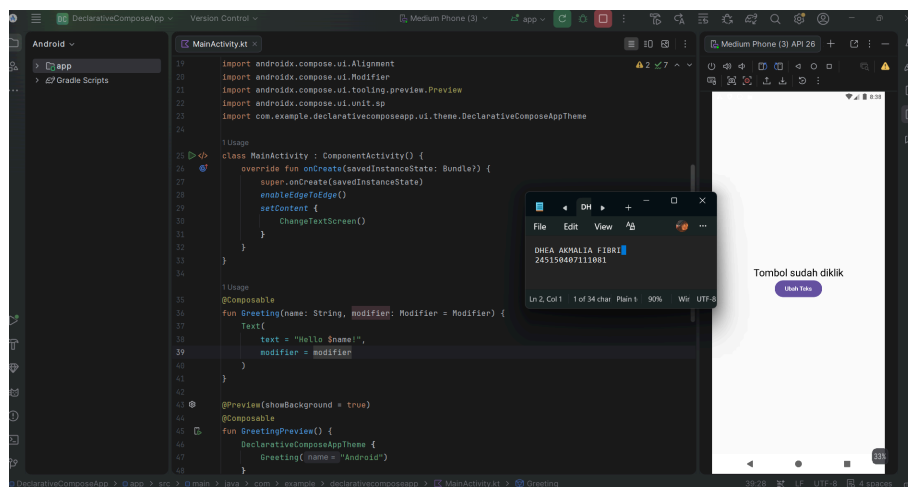
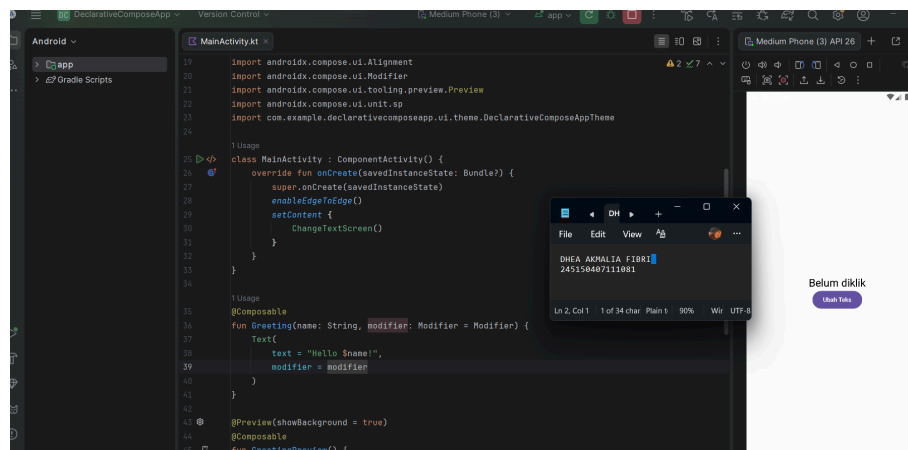


4. Pada fungsi onCreate(), ubah isi setContent agar memanggil ChangeTextScreen():

```
setContent {
    ChangeTextScreen()
}
```



5. Jalankan aplikasi. Pastikan hasil tampilannya sama dengan project XML/View.



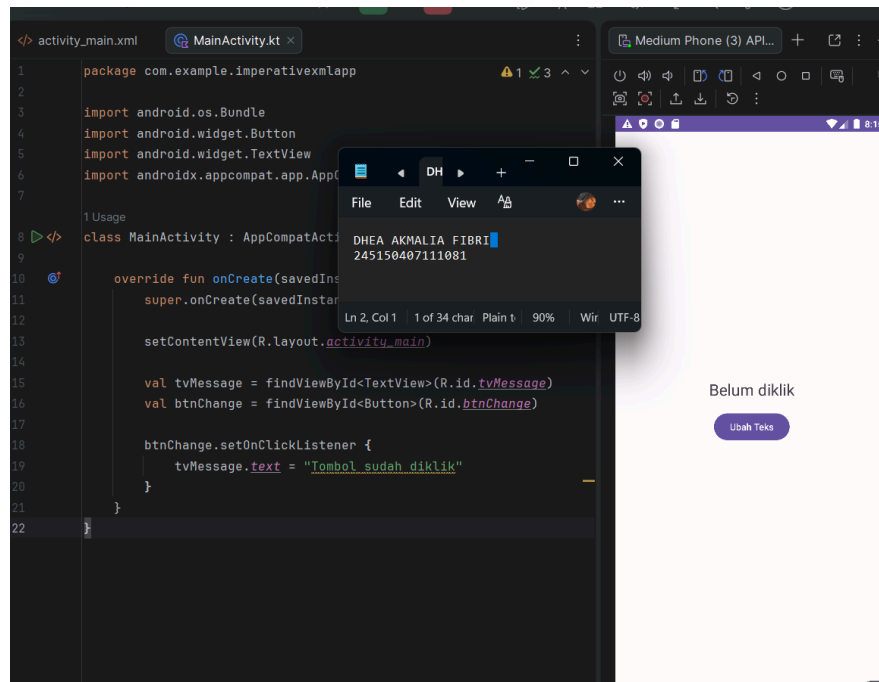
Aktivitas Percobaan

1. Jalankan kedua aplikasi dan ambil screenshot kondisi sebelum serta sesudah tombol diklik.

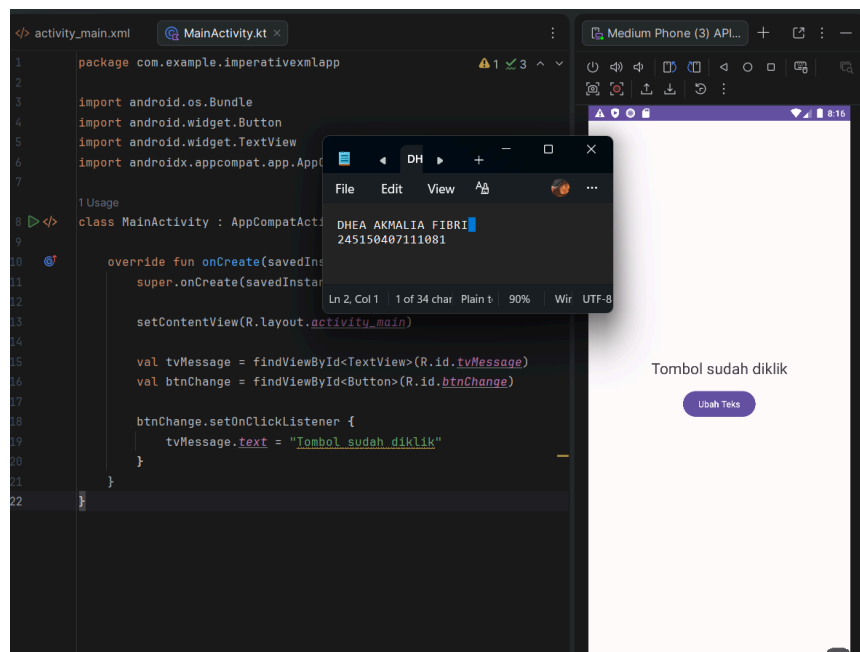
Jawab :

- Imperatif

Sebelum diklik :

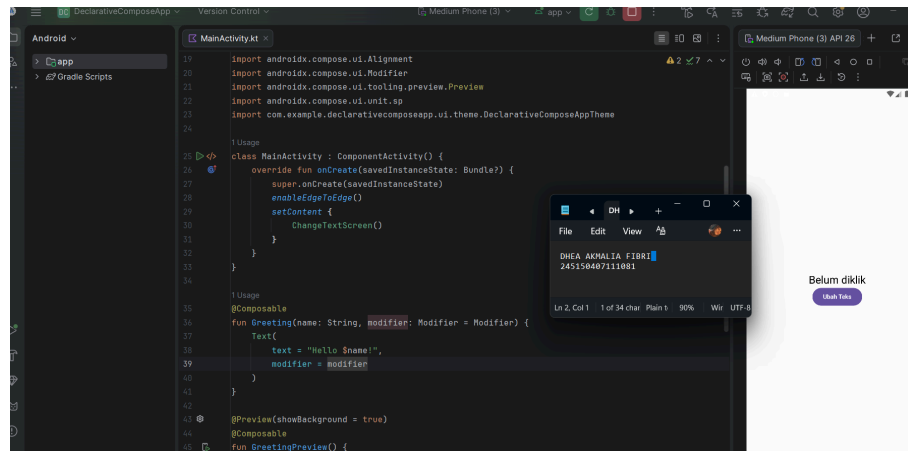


Sesudah diklik :

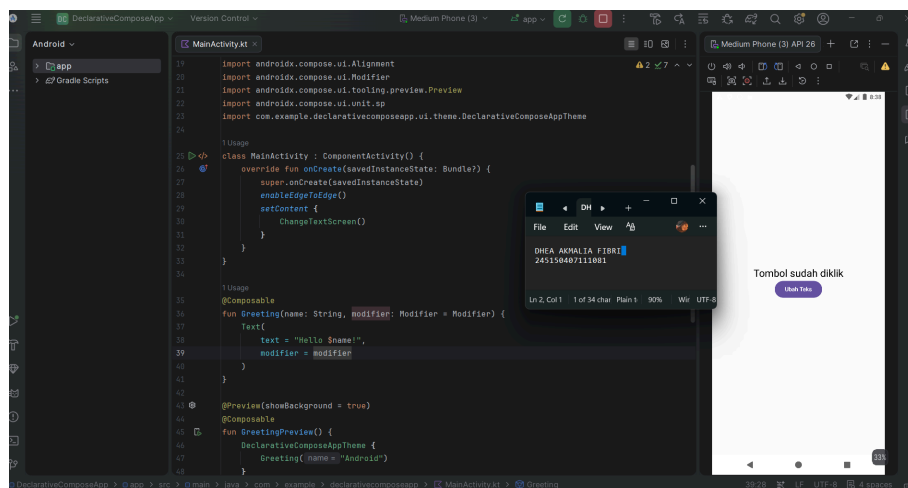


- Deklaratif

Sebelum diklik :



Sesudah diklik :

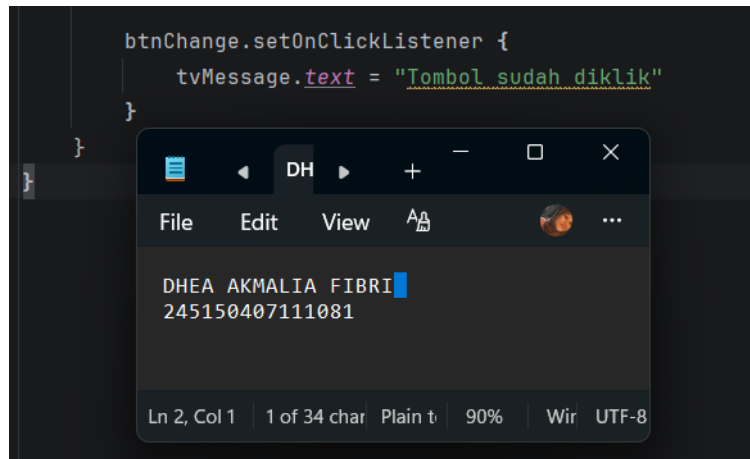


2. Pada project XML/View, objek apa yang diubah secara langsung ketika tombol diklik? Tunjukkan baris kodenya.

Jawab :

Objek yang diubah secara langsung ketika tombol diklik adalah TextView yang memiliki ID tvMessage. Ketika tombol btnChange diklik, program langsung mengubah nilai teks pada TextView tersebut, jadi program secara langsung memerintahkan View untuk mengubah tampilannya.

Perubahan tersebut terdapat di baris kode :

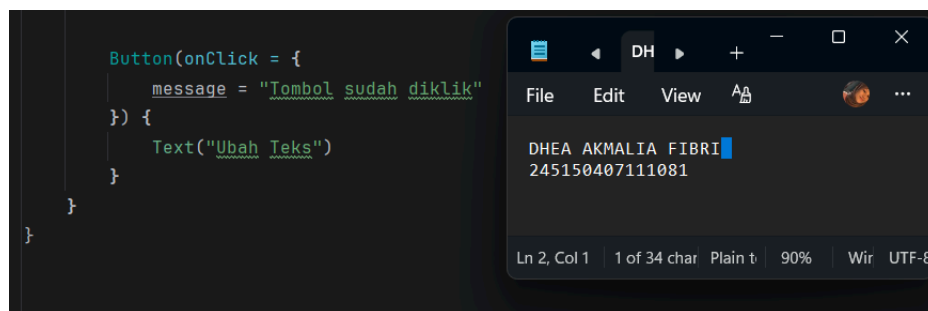


3. Pada project Compose, nilai apa yang diubah ketika tombol diklik? Tunjukkan baris kodenya.

Jawab :

Nilai yang diubah ketika tombol diklik adalah state message. Pada awalnya, message memiliki nilai “Belum diklik”. Ketika tombol ditekan, nilainya diubah menjadi “Tombol sudah diklik”.

Perubahan tersebut terdapat di baris kode :



4. Mengapa pada Compose tidak terdapat perintah seperti tvMessage.text atau setText()?

Jawab :

Pada compose tidak terdapat perintah seperti tvMessage.text atau setText() karena compose menggunakan pendekatan deklaratif yaitu merubah tampilan UI berdasarkan state yang sedang dimiliki. Jadi misalnya ketika state berubah, compose akan melakukan recomposition dan memperbarui bagian UI yang menggunakan state tersebut. Oleh karena itu, pada pendekatan deklaratif hanya perlu mengubah message, sedangkan perubahan pada teks di layar dilakukan oleh Compose.

5. Jelaskan alur perubahan pada XML/View dengan pola: event -> View -> perubahan tampilan.

Jawab :

Pada XML/View, proses dimulai ketika pengguna melakukan event, yaitu menekan tombol “Ubah Teks”, setelah event tersebut terjadi, program menggunakan View tvMessage yang sebelumnya sudah ditemukan dengan findViewById(). Selanjutnya, program secara langsung

mengubah properti text pada View tersebut menjadi “Tombol sudah diklik”. Oleh karena itu, teks yang terlihat pada layar ikut berubah.

6. Jelaskan alur perubahan pada Compose dengan pola: event -> state -> recomposition -> perubahan tampilan.

Jawab :

Pada Compose, proses dimulai ketika pengguna menekan tombol “Ubah Teks” sebagai sebuah event. Setelah tombol ditekan, program mengubah nilai state message dari “Belum diklik” menjadi “Tombol sudah diklik”. Karena message merupakan state yang diamati oleh Compose, perubahan tersebut menyebabkan recomposition. Compose kemudian memperbarui bagian UI yang menggunakan message, sehingga teks yang sebelumnya “Belum diklik” berubah menjadi “Tombol sudah diklik”.

7. Apa yang terjadi jika message diganti menjadi variabel biasa: `var message = "Belum diklik"`? Jelaskan penyebabnya.

Jawab :

Jika message diganti menjadi variabel biasa seperti `var message = "Belum diklik"`, perubahan nilainya tidak akan secara otomatis membuat tampilan UI diperbarui. Hal ini terjadi karena variabel biasa bukan state yang diamati oleh Compose. Walaupun nilai message dapat berubah ketika tombol diklik, Compose tidak mengetahui bahwa perubahan tersebut harus membuat UI digambar ulang. Selain itu, jika terjadi recomposition, nilai variabel biasa dapat kembali ke nilai awal. Oleh karena itu, pada Compose digunakan `remember` dan `mutableStateOf()` agar perubahan nilai dapat diketahui oleh Compose dan memicu recomposition.

8. Tuliskan kesimpulan perbedaan cara berpikir imperatif dan deklaratif menggunakan kalimat Anda sendiri.

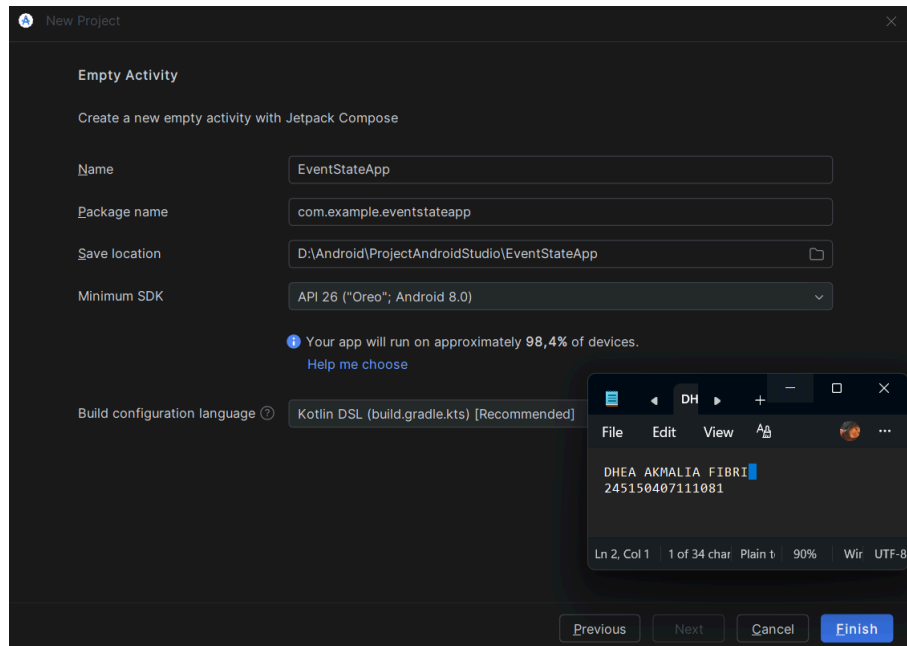
Jawab :

Menurut pemahaman saya, perbedaan utama dari pendekatan imperatif dan deklaratif terletak pada cara program menangani perubahan tampilan. Pada pendekatan imperatif seperti XML/View, secara langsung menentukan View yang harus diubah ketika terjadi suatu event, misalnya mengubah teks pada TextView. Sedangkan pada pendekatan yang deklaratif, fokusnya pada state atau kondisi yang sedang terjadi. Ketika statenya berubah, Compose akan memperbarui UI yang berkaitan dengan state tersebut melalui recomposition. Jadi pada intinya, imperatif berfokus pada bagaimana dan bagian mana UI yang akan diubah, sedangkan deklaratif fokusnya terhadap kondisi atau state yang menjadi dasar tampilan UI.

Event Handling State

Pra-Percobaan

1. Buat sebuah Project Android Studio baru, dengan Template Empty Activity untuk Phone and Tablet. Pastikan project yang dibuat menggunakan Jetpack Compose. Tunggu sampai proses Build inisialisasi project baru ini selesai dilakukan sebelum melanjutkan percobaan ke langkah selanjutnya.



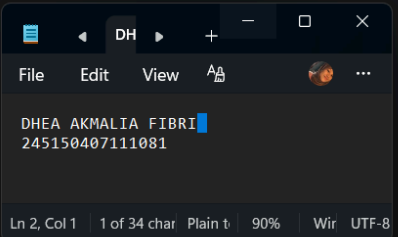
2. Buka file MainActivity.kt dan tambahkan fungsi composable berikut ini di luar class MainActivity:

```
@Composable
fun EventStateTest() {
    var count = 0
    Column(
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally,
        modifier = Modifier.fillMaxSize()
    ) {
        Text("Counter : $count")
        Button(onClick = {
            count++
            Log.d("TEST", "Counter berubah : $count")
        }) {
            Text("Tambah")
        }
    }
}
```

```

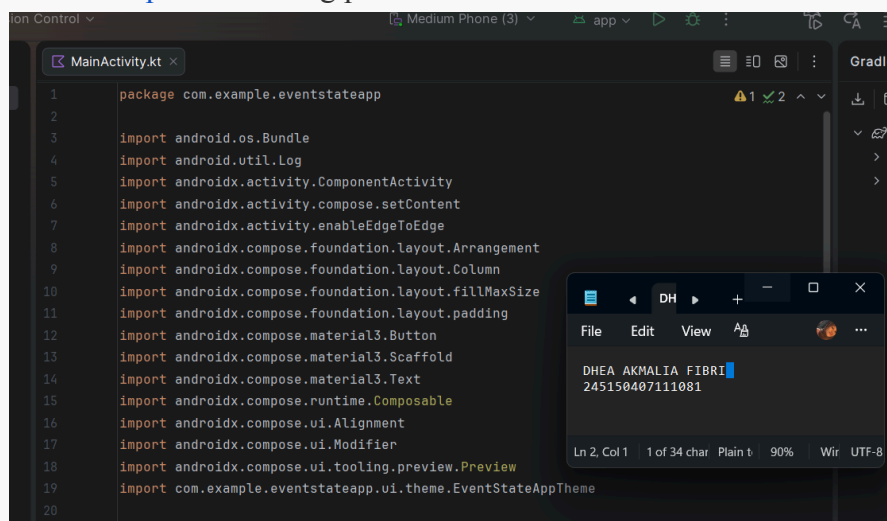
51     }
52 }
53
54 @Composable
55 fun EventStateTest() {
56     var count = 0
57
58     Column(
59         verticalArrangement = Arrangement.Center,
60         horizontalAlignment = Alignment.CenterHorizontally,
61         modifier = Modifier.fillMaxSize()
62     ) {
63         Text("Counter: $count")
64
65         Button(onClick = {
66             count++
67             Log.d( tag = "TEST", msg = "Counter berubah: $count")
68         }) {
69             Text("Tambah")
70         }
71     }
72 }

```



3. Import semua keyword dan konstruk atas kesalahan sintaksis dan struktur data yang muncul sebagai akibat dari penambahan kode program yang baru ditambahkan tersebut, termasuk namun tidak terbatas pada:

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.material3.Button
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import [androidx.compose.ui.tooling.preview.Preview](#)



```

1 package com.example.eventstateapp
2
3 import android.os.Bundle
4 import android.util.Log
5 import androidx.activity.ComponentActivity
6 import androidx.activity.compose.setContent
7 import androidx.activity.enableEdgeToEdge
8 import androidx.compose.foundation.layout.Arrangement
9 import androidx.compose.foundation.layout.Column
10 import androidx.compose.foundation.layout.fillMaxSize
11 import androidx.compose.foundation.layout.padding
12 import androidx.compose.material3.Button
13 import androidx.compose.material3.Scaffold
14 import androidx.compose.material3.Text
15 import androidx.compose.runtime.Composable
16 import androidx.compose.ui.Alignment
17 import androidx.compose.ui.Modifier
18 import androidx.compose.ui.tooling.preview.Preview
19 import com.example.eventstateapp.ui.theme.EventStateAppTheme
20

```

4. Buka file MainActivity.kt, temukan fungsi onCreate() yang telah disediakan oleh template aplikasi Android di Android Studio. Ubah isi setContent{ } menjadi

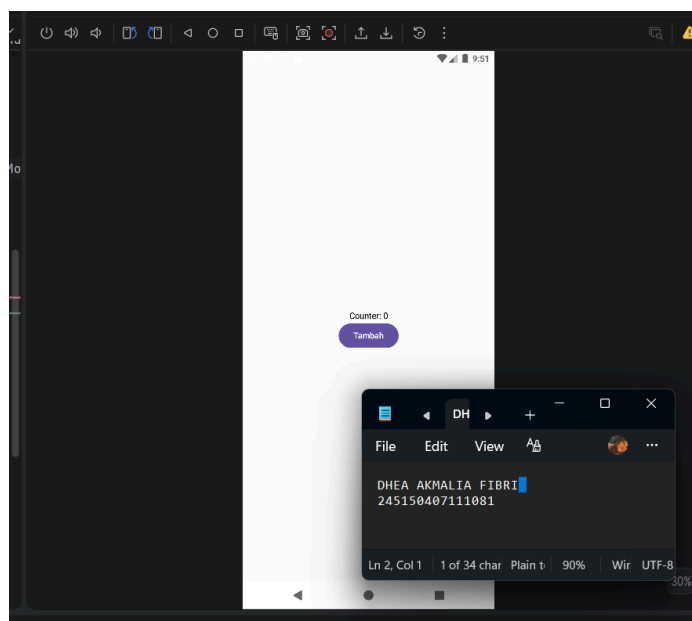
```
setContent {  
    EventStateTest()  
}
```

5. Sebelum menjalankan aktivitas percobaan, pastikan bahwa tidak ada penulisan kode program yang salah yang dapat mengakibatkan program tidak dapat dijalankan dengan baik pada saat di-compile.

Aktivitas Percobaan

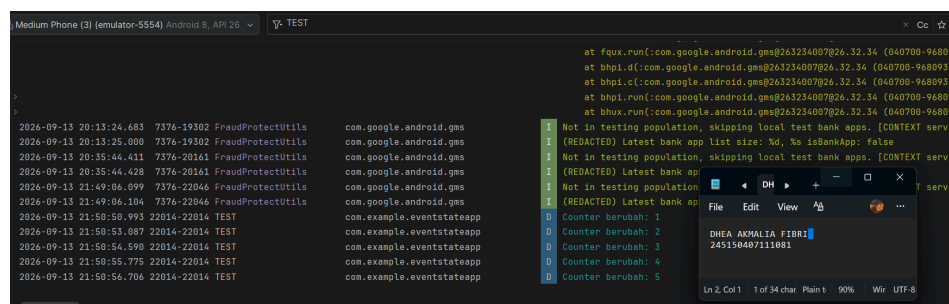
1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.

Jawab :



2. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?

Jawab :



3. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?

Jawab :

Pada tampilan aplikasi, nilai Counter tetap menunjukkan angka 0 setiap kali tombol “Tambah” diklik. Meskipun tombol sudah ditekan beberapa kali dan nilai count sebenarnya bertambah, perubahan tersebut tidak terlihat pada tampilan aplikasi. Hal ini terlihat berbeda dengan Logcat yang menunjukkan bahwa nilai counter berubah setiap kali tombol ditekan.

4. Apa yang menyebabkan hal tersebut terjadi?

Jawab :

Hal tersebut terjadi karena count masih menggunakan variabel biasa, yaitu `var count = 0`, bukan state Compose. Ketika tombol diklik, `count++` memang menambah nilai count, tetapi Compose tidak mengetahui bahwa nilai tersebut berubah sehingga tidak melakukan recomposition pada tampilan. Oleh karena itu, teks “Counter : \$count” pada layar tidak diperbarui dan tetap menampilkan Counter : 0. Inilah yang menjadi tujuan percobaan ini, yaitu melihat bahwa perubahan variabel biasa tidak otomatis menyebabkan UI Compose diperbarui.

5. Langkah selanjutnya adalah mengubah kode sebelumnya untuk menggunakan state yaitu pada Composable EventStateTest.
6. Ubah kode program pada bagian fungsi Composable EventStateTest dari yang sebelumnya:

```
var count = 0
```

menjadi:

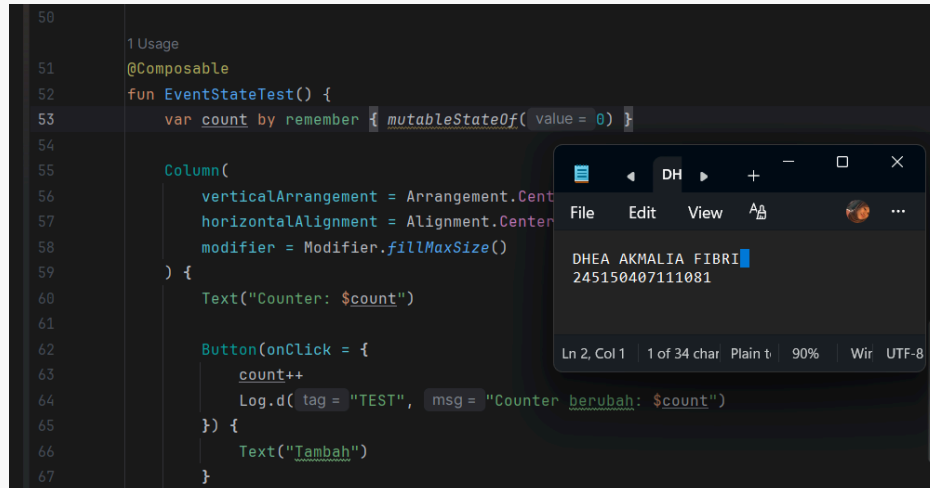
```
var count by remember { mutableStateOf(0) }
```

7. Berikut kode lengkap EventStateTest:

```
@Composable
fun EventStateTest() {
    var count by remember { mutableStateOf(0) }
    Column(
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally,
        modifier = Modifier.fillMaxSize()
    ) {
        Text("Counter : $count")
        Button(onClick = {
            count++
            Log.d("TEST", "Counter berubah : $count")
        }) {
            Text("Tambah")
        }
    }
}
```

```
}  
}  
}
```

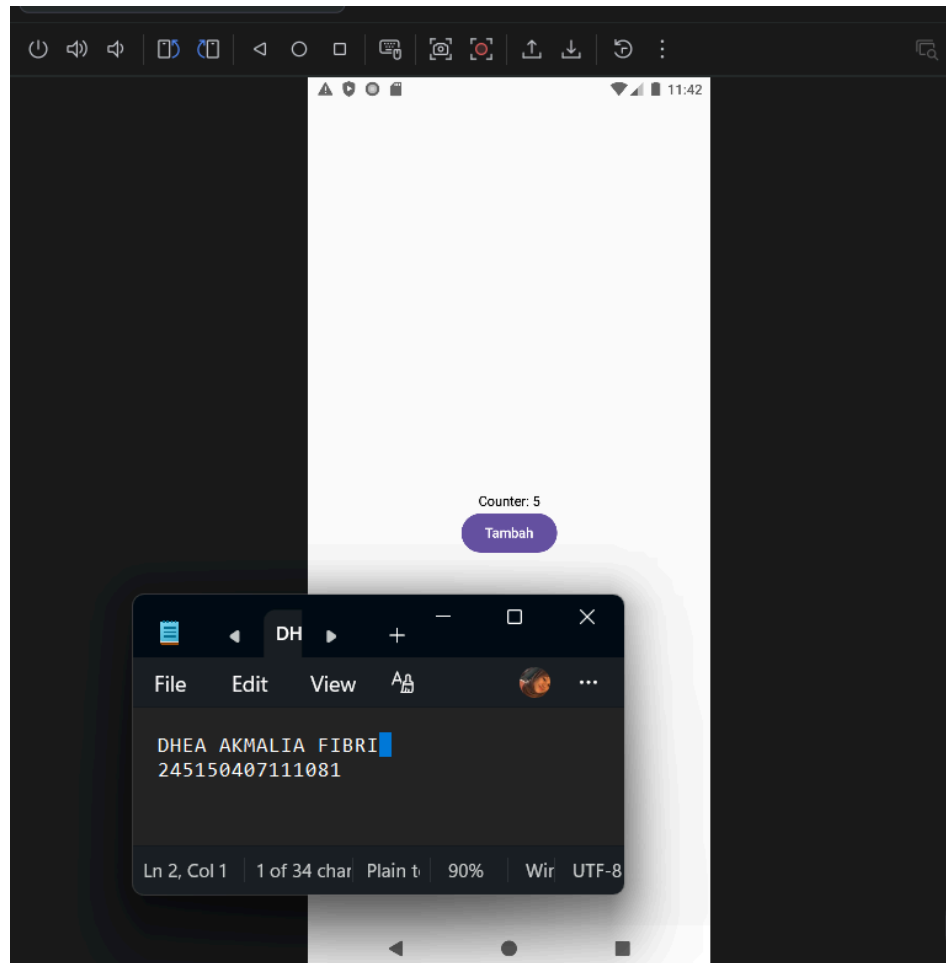
Jawab :



```
50  
51  
52  
53  
54  
55  
56  
57  
58  
59  
60  
61  
62  
63  
64  
65  
66  
67  
50  
1 Usage  
51  
@Composable  
52  
fun EventStateTest() {  
53  
    var count by remember { mutableStateOf(value = 0) }  
54  
55  
    Column(  
56  
        verticalArrangement = Arrangement.Center  
57  
        horizontalAlignment = Alignment.Center  
58  
        modifier = Modifier.fillMaxSize()  
59  
    ) {  
60  
        Text("Counter: $count")  
61  
62  
        Button(onClick = {  
63  
            count++  
64  
            Log.d( tag = "TEST", msg = "Counter berubah: $count")  
65  
        }) {  
66  
            Text("Tambah")  
67  
        }  
    }  
}
```

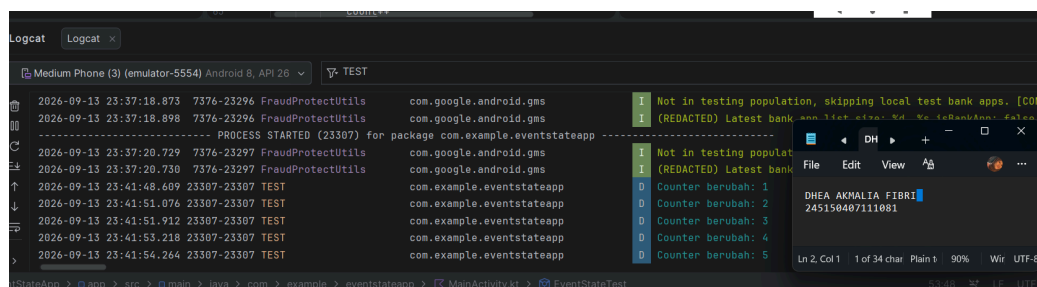
8. Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.

Jawab :



9. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?

Jawab :



10. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?

Jawab :

Pada tampilan aplikasi, nilai Counter ikut bertambah setiap kali tombol “Tambah” diklik, misalnya awalnya Counternya 0, setelah tombol diklik sekali Counternya menjadi satu, kemudian jika diklik sekali lagi dan seterusnya counternya terus bertambah.

11. Apa yang menyebabkan hal tersebut terjadi dan berbeda dengan percobaan sebelumnya?

Jawab :

Hal tersebut terjadi karena pada percobaan ini count sudah menggunakan state dengan `remember { mutableStateOf(0) }`. Ketika tombol diklik, nilai count berubah melalui `count++`, dan Compose mengetahui bahwa state tersebut mengalami perubahan sehingga UI yang menggunakan count diperbarui. Pada percobaan sebelumnya, count hanya merupakan variabel biasa sehingga perubahan nilainya tidak membuat Compose memperbarui tampilan secara otomatis.

12. Apa yang dimaksud recomposition pada Compose?

Jawab :

Recomposition adalah proses ketika Compose menjalankan kembali bagian UI yang menggunakan state yang berubah agar tampilan sesuai dengan nilai state terbaru. Ketika nilai count berubah setelah tombol “Tambah” diklik, Compose melakukan recomposition sehingga teks Counter: \$count diperbarui dan menampilkan angka terbaru.

13. Langkah selanjutnya adalah menambah kode sebelumnya dengan menambah state baru yaitu text bertipe string pada Composable EventStateTest. Kemudian di dalam Column panggil composable OutlinedTextField di bawah Button Tambah. Penambahan kode program membuat kode pada fungsi Composable EventStateTest menjadi:

```
fun EventStateTest() {
    var count by remember { mutableStateOf(0) }
    var text by remember { mutableStateOf("") }

    Column(
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally,
        modifier = Modifier.fillMaxSize()
    ) {
        Text("Counter : $count")
        Button(onClick = {
            count++
            Log.d("TEST", "Counter berubah : $count")
        }) {
            Text("Tambah")
        }
        OutlinedTextField(
            value = text,
            onChange = {
                text = it
                Log.d("TEST", "Text berubah : $it")
            },
            label = { Text("Text Field") }
```

```

    )
}
}

```

Jawab :

The screenshot shows an IDE with a file named `MainActivity.kt`. The code is as follows:

```

1 Usage
21 @Composable
22 fun EventStateTest() {
23     var count by remember { mutableStateOf( value = 0) }
24     var text by remember { mutableStateOf( value = "") }
25
26     Column(
27         verticalArrangement = Arrangement.Center,
28         horizontalAlignment = Alignment.CenterHorizontally,
29         modifier = Modifier.fillMaxSize()
30     ) {
31         Text("Counter: $count")
32
33         Button(onClick = {
34             count++
35             Log.d( tag = "TEST", msg = "Counter berubah: $count")
36         }) {
37             Text("Tambah")
38         }
39     }
40 }

```

Overlaid on the code is a LogCat window titled "DH" showing a log message:

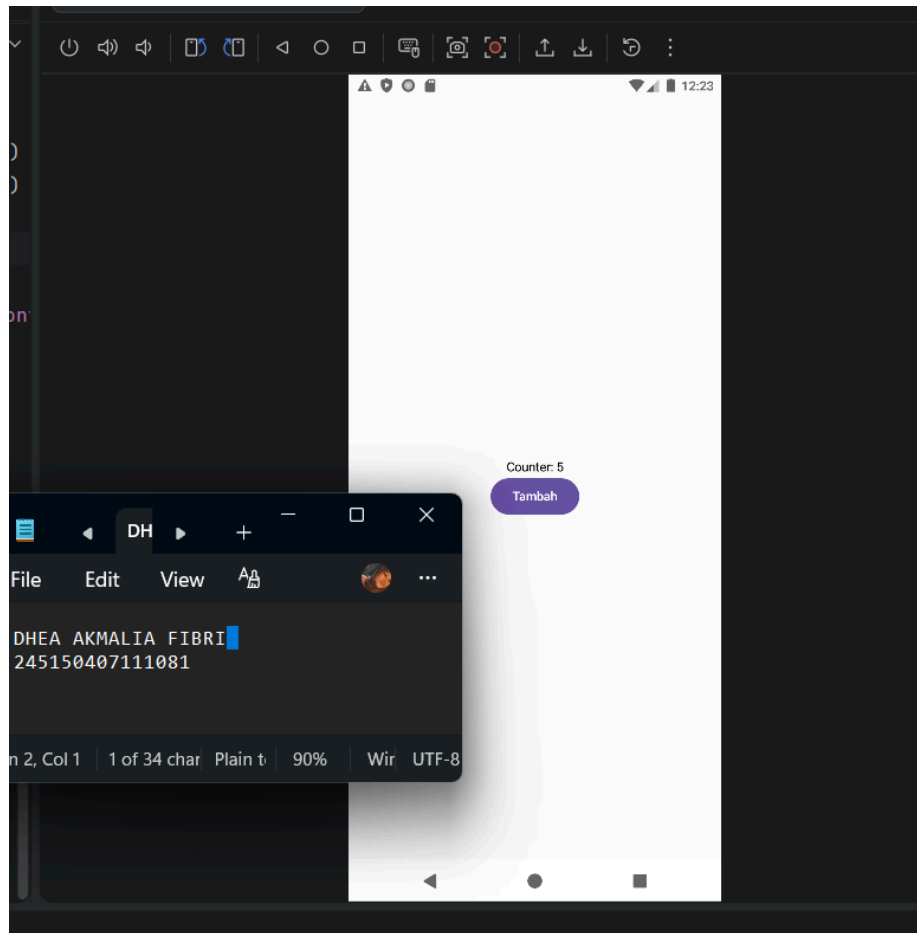
```

DHEA AKMALIA FIBRI
245150407111081
Ln 2, Col 1 1 of 34 char Plain t 90% Wir UTF-8

```

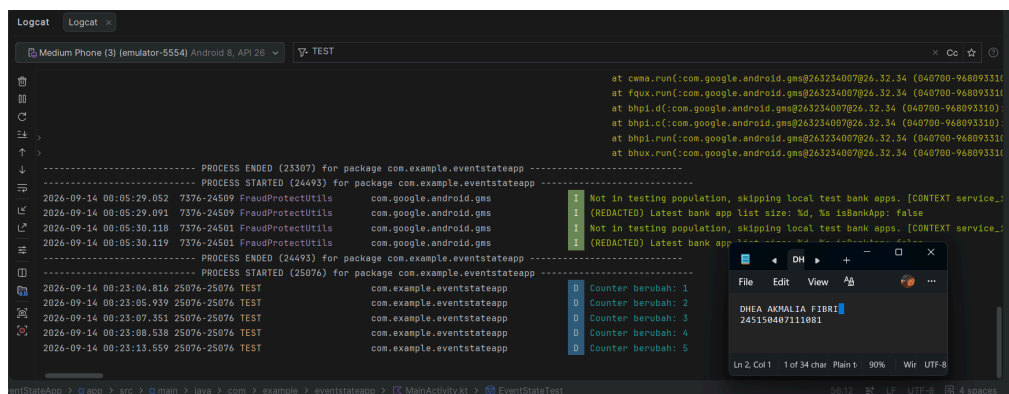
14. Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.

Jawab :



15. Apa yang muncul di LogCat setiap kali menginputkan huruf di Text Field?

Jawab :



16. Apa yang terjadi pada tampilan aplikasi setiap kali menginputkan huruf di Text Field?

Jawab :

Pada tampilan aplikasi, setiap huruf yang diketik pada Text Field tersebut. Misalnya ketika mengetik “Halo”, teks yang terlihat di Text Field juga menjadi “Halo”. Hal ini terjadi karena nilai text digunakan sebagai value pada OutlinedTextField dan diperbarui setiap kali pengguna memasukkan atau mengubah huruf.

17. Apa yang menyebabkan hal tersebut terjadi?

Jawab :

Hal tersebut terjadi karena teks yang dimasukkan pengguna disimpan sebagai state text menggunakan `remember {mutableStateOf(“ “)}`. Ketika pengguna mengetik atau mengubah isi Text Field event `onValueChange` dijalankan dan nilai text diperbarui. Karena text merupakan state yang diamati oleh Compose, perubahan tersebut menyebabkan Compose melakukan recomposition sehingga tampilan Text Field diperbarui sesuai dengan teks terbaru.

18. Sebutkan state apa saja yang ada pada program yang terakhir dibuat?

Jawab :

Pada program terakhir terdapat dua state, yaitu count dan text. State count digunakan untuk menyimpan nilai jumlah counter yang bertambah ketika tombol “Tambah” diklik, sedangkan state text digunakan untuk menyimpan teks yang dimasukkan pengguna melalui Text Field.

19. Sebutkan event apa saja yang ada pada program yang terakhir dibuat?

Jawab :

Pada program terakhir terdapat dua event, yaitu event `onClick` pada tombol “Tambah” dan event `onValueChange` pada `OutlinedTextField`. Event `onClick` digunakan untuk menangani aksi ketika tombol “Tambah”, sedangkan `onValueChange` digunakan untuk menangani perubahan ketika pengguna memasukkan atau mengubah teks pada Text Field.

20. Kapan event - event tersebut dijalankan?

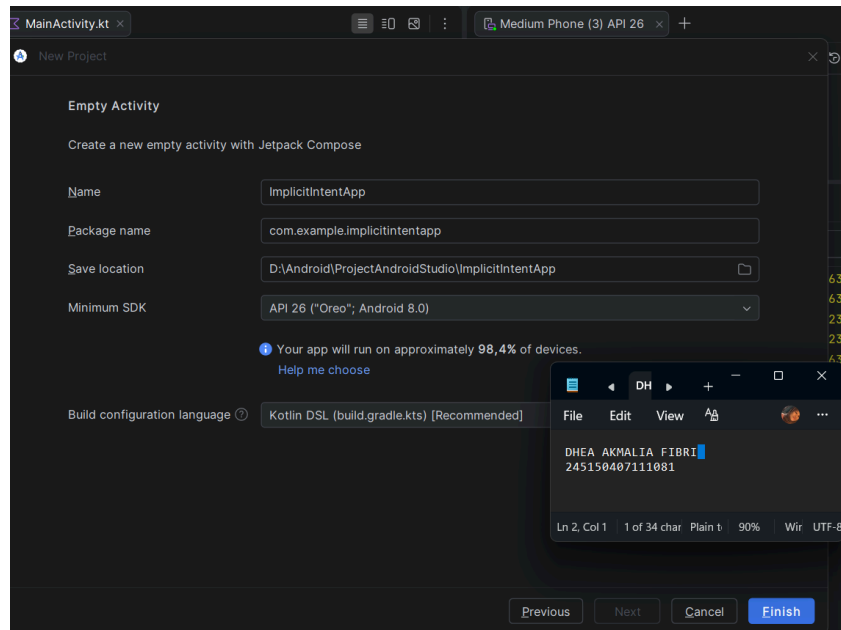
Jawab :

Event `onClick` dijalankan setiap kali pengguna menekan tombol “Tambah”, sehingga nilai count akan bertambah. Sedangkan event `onValueChange` dijalankan setiap kali pengguna memasukkan atau mengubah huruf pada Text Field, sehingga nilai text ikut diperbarui. Dengan demikian, kedua event tersebut dijalankan ketika pengguna melakukan interaksi langsung dengan komponen yang bersangkutan.

Implisit Intent

Pra-Percobaan

1. Buat sebuah Project Android Studio baru, dengan Template Empty Activity untuk Phone and Tablet. Pastikan project yang dibuat menggunakan Jetpack Compose. Tunggu sampai proses Build inisialisasi project baru ini selesai dilakukan sebelum melanjutkan percobaan ke langkah selanjutnya.



2. Buka file MainActivity.kt dan tambahkan fungsi composable berikut ini di luar class MainActivity

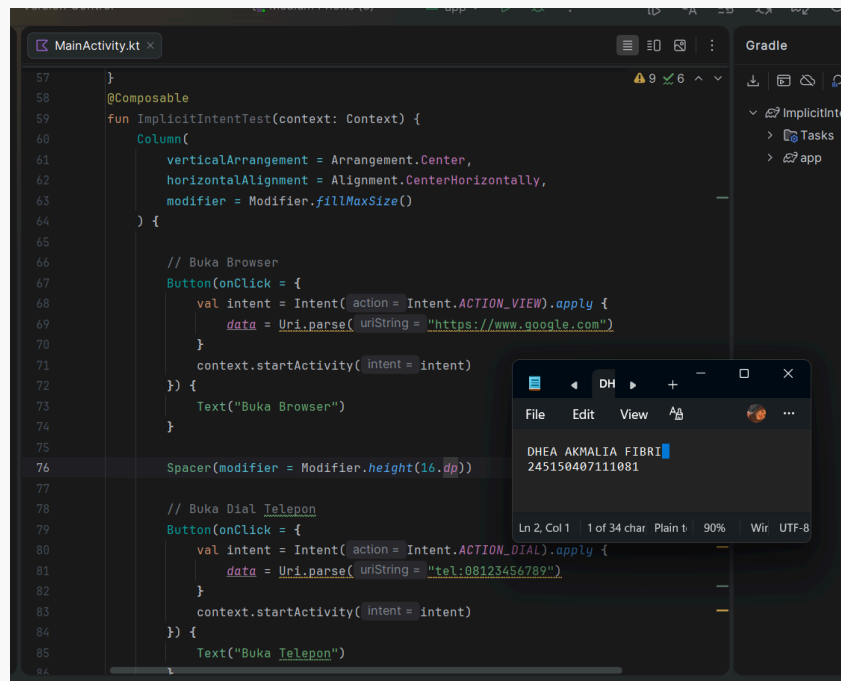
@Composable

```
fun ImplicitIntentTest(context: Context) {
    Column(
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally,
        modifier = Modifier.fillMaxSize()
    ) {
        // Buka Browser
        Button(onClick = {
            val intent = Intent(Intent.ACTION_VIEW).apply {
                data = Uri.parse("https://www.google.com")
            }
            context.startActivity(intent)
        }) {
            Text("Buka Browser")
        }
        Spacer(modifier = Modifier.height(16.dp))
        // Buka Dial Telepon
        Button(onClick = {
            val intent = Intent(Intent.ACTION_DIAL).apply {
                data = Uri.parse("tel:08123456789")
            }
            context.startActivity(intent)
        }) {
            Text("Buka Telepon")
        }
    }
}
```

```

    }
    Spacer(modifier = Modifier.height(16.dp))
    // Kirim Email (dengan chooser)
    Button(onClick = {
        val intent = Intent(Intent.ACTION_SENDTO).apply {
            data = Uri.parse("mailto:test@gmail.com")
        }
        val chooser = Intent.createChooser(intent, "Pilih aplikasi email")
        context.startActivity(chooser)
    }) {
        Text("Kirim Email")
    }
    Spacer(modifier = Modifier.height(16.dp))
    // Buka Maps
    Button(onClick = {
        val intent = Intent(Intent.ACTION_VIEW).apply {
            data = Uri.parse("geo:-7.250445,112.768845")
        }
        context.startActivity(intent)
    }) {
        Text("Buka Maps")
    }
}

```

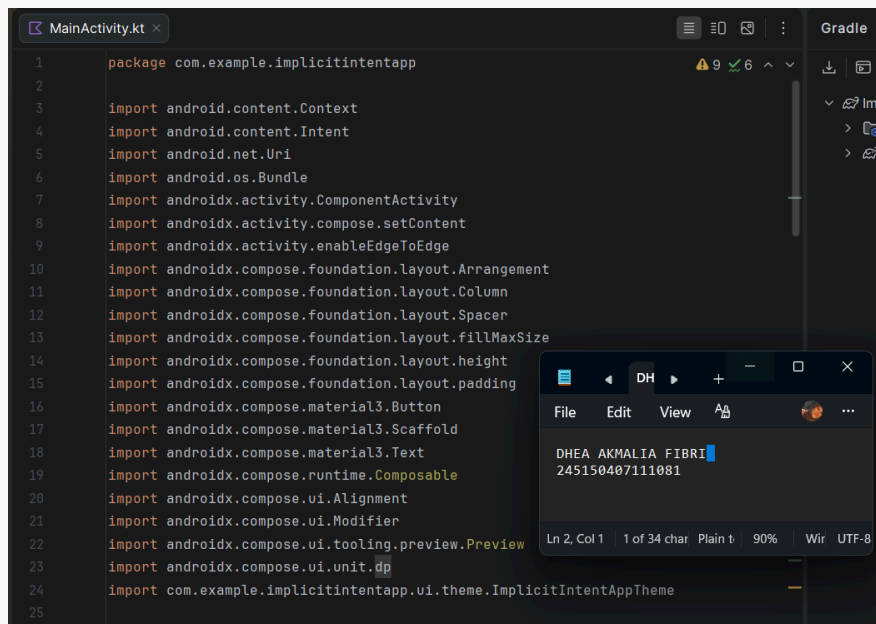


3. Import semua keyword dan konstruk atas kesalahan sintaksis dan struktur data yang muncul sebagai akibat dari penambahan kode program yang baru ditambahkan tersebut, termasuk namun tidak terbatas pada:

```

import android.content.Context
import android.content.Intent
import android.net.Uri
import androidx.compose.foundation.layout.*
import androidx.compose.material3.Button
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp

```

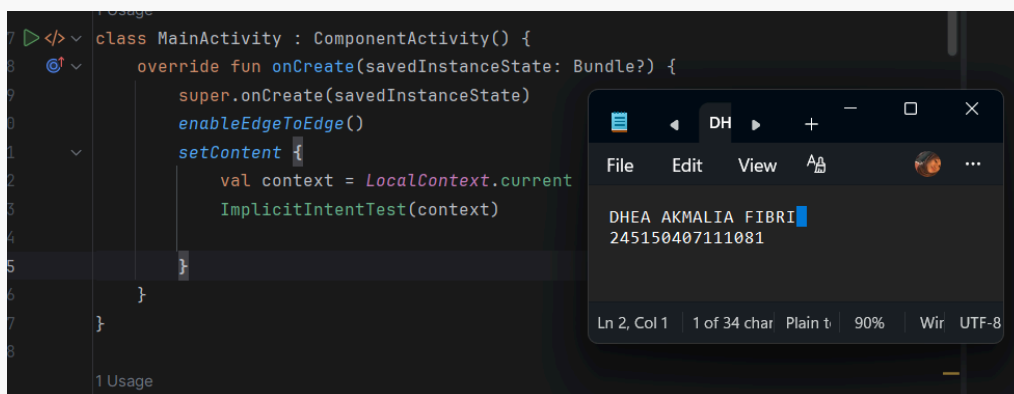


4. Buka file MainActivity.kt, temukan fungsi onCreate() yang telah disediakan oleh template aplikasi Android di Android Studio. Ubah isi setContent{ } menjadi

```

setContent {
    val context = LocalContext.current
    ImplicitIntentTest(context)
}

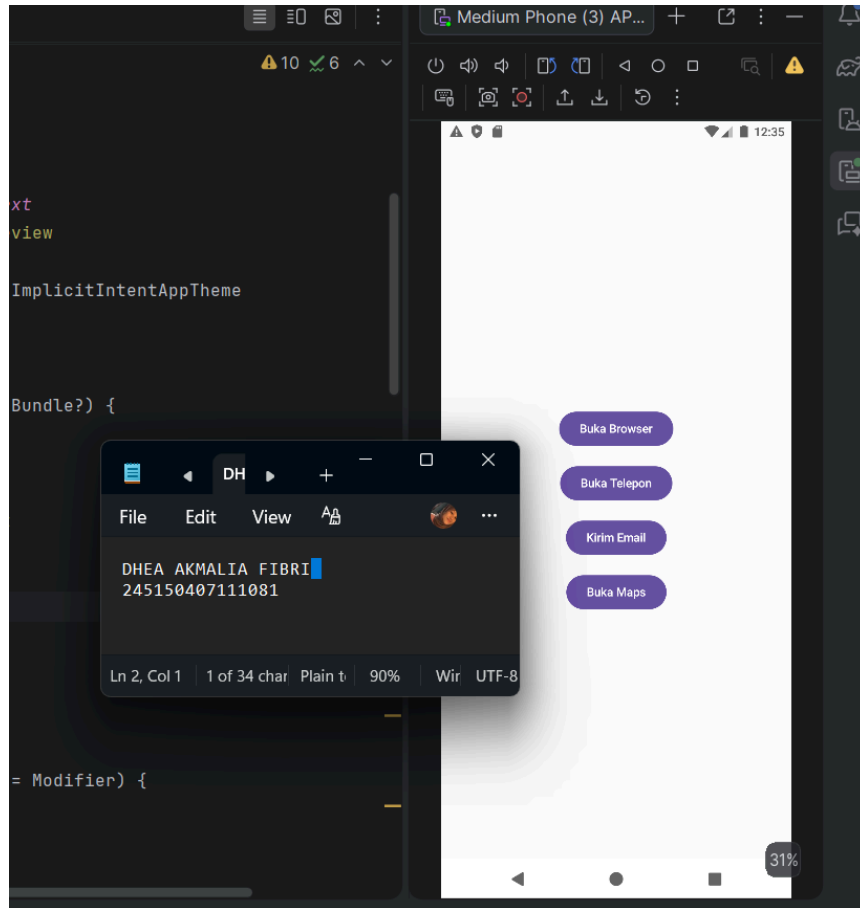
```



Aktivitas Percobaan

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Lakukan pengujian pada setiap tombol:

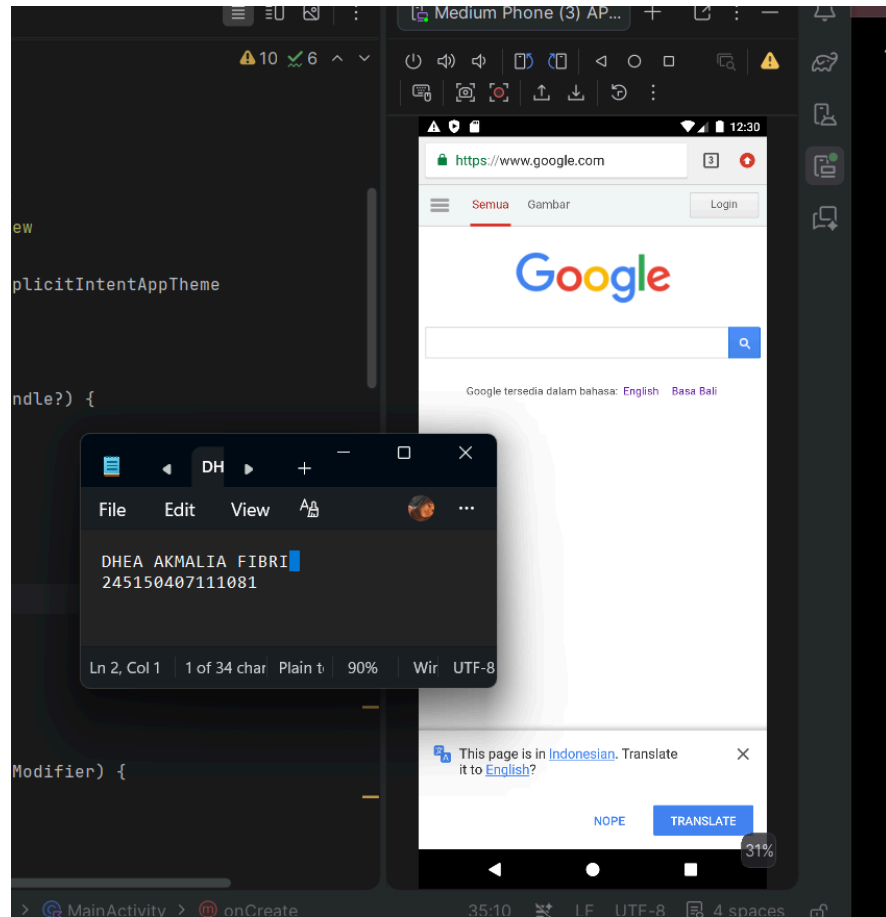
Jawab :



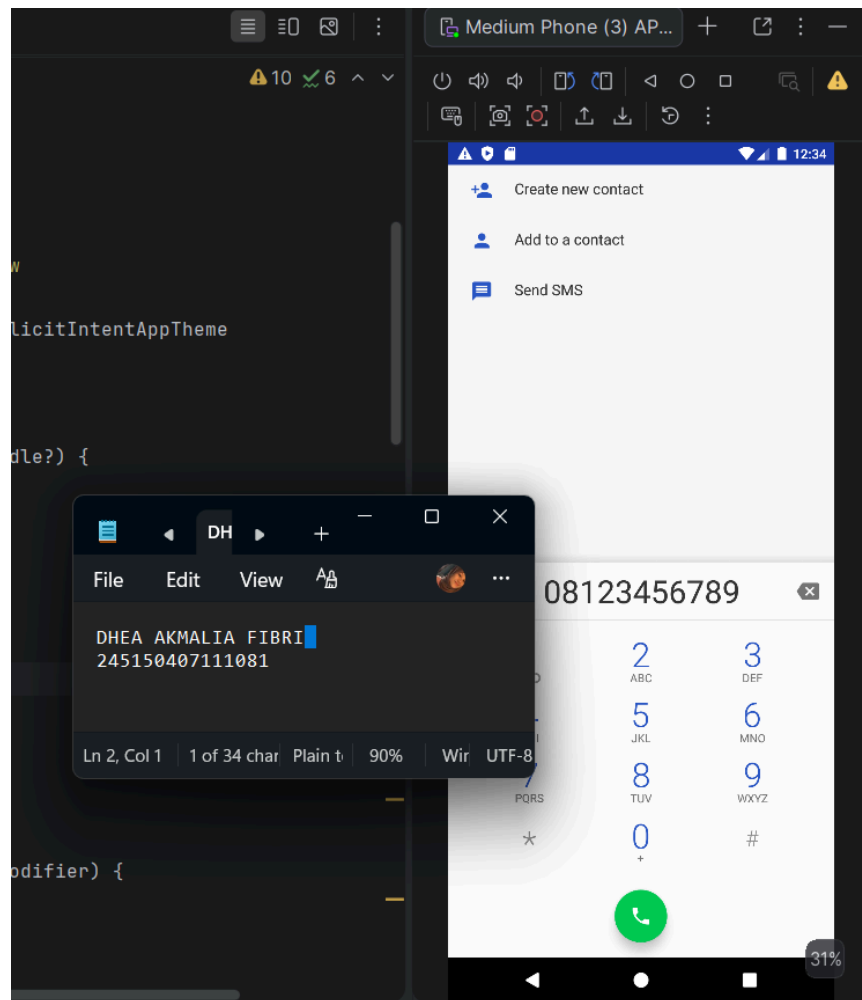
2. Klik Buka Browser
3. Klik Buka Telepon
4. Klik Kirim Email
5. Klik Buka Maps
6. Amati apa yang terjadi pada setiap aksi, kemudian ambil screenshot hasilnya

Jawab :

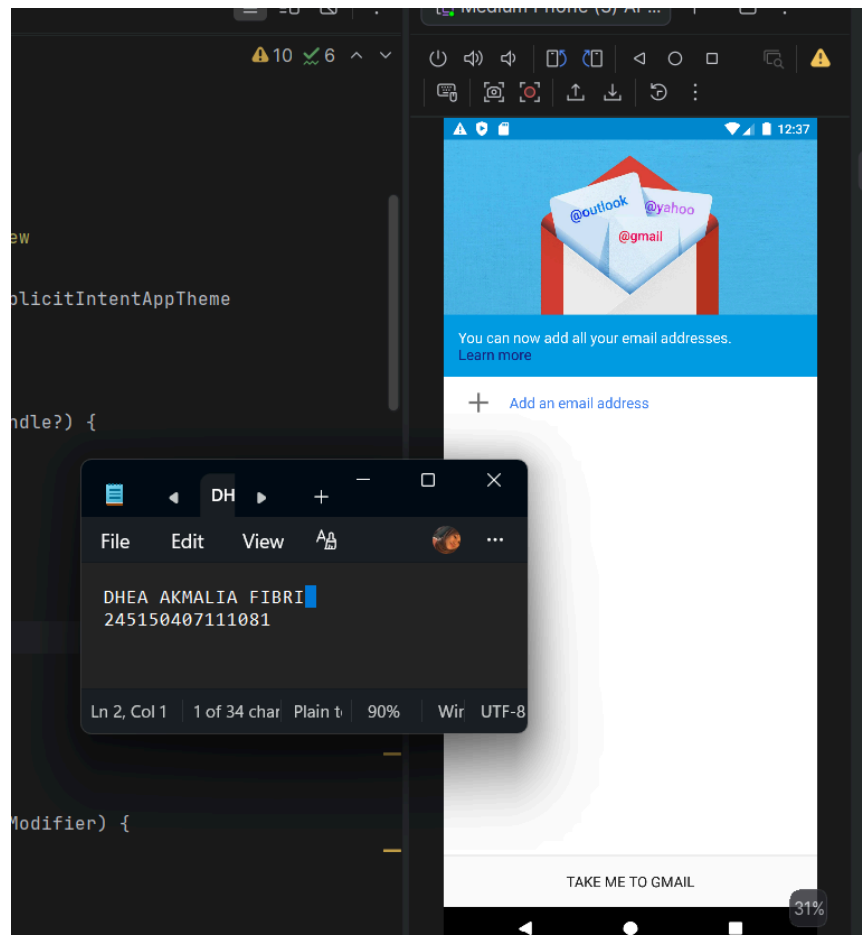
- Klik buka browser



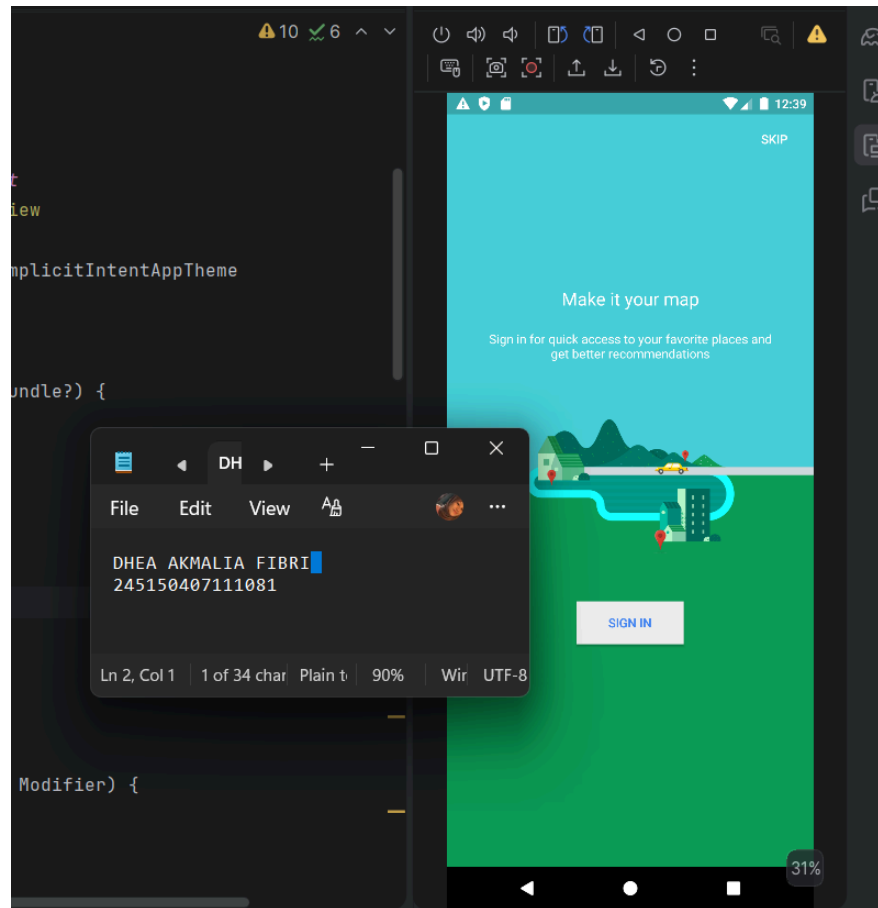
- Klik buka telepon



- Klik kirim email



- Klik buka maps



7. Pada kode yang digunakan, terdapat beberapa action seperti:
8. Intent.ACTION_VIEW
9. Intent.ACTION_DIAL
10. Intent.ACTION_SENDTO
11. Apa perbedaan fungsi dari masing-masing action intent tersebut?

Jawab :

- Intent.ACTION_VIEW : digunakan untuk membuka atau menampilkan suatu data menggunakan aplikasi yang sesuai.
- Intent.ACTION_DIAL: digunakan untuk membuka aplikasi telepon pada halaman dial atau panggilan dengan nomor telepon yang sudah diberikan .
- Intent.ACTION_SENDTO: digunakan untuk mengirim data kepada tujuan tertentu.

12. Pada kode yang digunakan, terdapat beberapa data seperti:
13. data = Uri.parse("mailto:test@gmail.com")
14. data = Uri.parse("<https://www.google.com>")
15. data = Uri.parse("geo:-7.250445,112.768845")
16. Apa fungsi data pada intent ?

Jawab :

Data berfungsi untuk menentukan data atau tujuan yang akan diproses oleh intent. Pada kode terdapat tiga jenis data yaitu diantaranya :

- <mailto:test@gmail.com> digunakan untuk menentukan alamat email tujuan yang akan digunakan ketika mengirim email.
- <https://www.google.com> digunakan untuk menentukan alamat website yang akan dibuka melalui browser.
- Geo:-7.250445,112.768845 digunakan untuk menentukan lokasi berdasarkan koordinat yang akan dibuka melalui aplikasi Maps. Jadi, isi data menentukan informasi apa yang akan diproses oleh Android sesuai dengan action yang digunakan.

17. Apa hubungan Action dan Data pada pilihan aplikasi yang dimunculkan oleh sistem Android

Jawab :

Hubungannya adalah Action menentukan apa yang ingin dilakukan, sedangkan Data menentukan terhadap data atau tujuan apa tindakan tersebut dilakukan. Kombinasi keduanya membantu sistem Android menentukan aplikasi yang sesuai untuk menjalankan Intent.